Minor Project Report

on

MULTI-USER CHATROOM APPLICATION

PulseChat

Submitted to:

Dr. Gagandeep Kaur-E12617

Associate Professor

University Institute of Computing

Submitted by:

Asmi Choudhary (25 MCA20325)

Master of Computer Applications (MCA)

Chandigarh University, Mohali

BONAFIDE CERTIFICATE

Certified that this project report "Multi-user Chatroom Application" is the bona fide work of "Asmi Choudhary" who carried out the project work under my/our supervision.

This project report is submitted in partial fulfillment of the requirements for the award of the degree of Master of Computer Applications (MCA).

SIGNATURE	SIGNATURE
Dr. Amanpreet Kaur Sandhu	Dr. Gagandeep Kaur-E12617
HEAD OF THE DEPARTMENT	Associate Professor
University Institute of Computing	University Institute of Computing

INTERNAL EXAMINER **EXTERNAL EXAMINER**

Signature and examination details are to be completed by the institution.

Table Of Contents

CHAPTER 1: INTRODUCTION 6-17
1.1 Need Identification 6
1.2 Identification of Problem 8
1.3 Identification of Tasks 10
1.4 Timeline 13
1.5 Organization of the Report 15
CHAPTER 2: BACKGROUND STUDY 18-36
2.1 Timeline of the Reported Problem 18
2.2 Proposed Solution 20
2.3 Bibliometric Analysis 23
2.4 Review Summary 28
2.5 Problem Definition 29
2.6 Goals/Objectives 30
2.7 Why Use These Technologies Over Others 32
CHAPTER 3: DESIGN AND PROCESS 37-59
3.1 Evaluation and Selection of Specifications/Features 37
3.2 Design Constraints 40
3.3 Design Flow 44
3.4 Creation of the Chatroom Application 51
3.5 Logo Designing and Naming 55
3.6 Implementation Plan/Methodology 56
CHAPTER 4: RESULT ANALYSIS AND VALIDATION 60-64
4.1 Implementation of Solution 60
4.2 Validation 63
CHAPTER 5: CONCLUSION AND FUTURE WORK 65-70
5.1 Conclusion 65
5.2 Future Work 66
REFERENCES 71
PLAGIARISM REPORT 72

Detailed Contents

3.1.1 Functional Specifications 37
3.1.2 Non-Functional Specifications 38
3.1.3 Evaluation and Selection of Technologies 38
3.1.4 Selection of Core Features 39
3.1.5 Data Management and Security Considerations 39
3.2.1 Technology Constraints 41
3.2.2 Database Constraints 41
3.2.3 Real-Time Communication Constraints 41
3.2.4 Security Constraints 42
3.2.5 Performance Constraints 42
3.2.6 User Interface and Usability Constraints 42
3.2.7 Development Time and Resource Constraints 43
3.2.8 Maintainability and Scalability Constraints 43
3.3.1 Frontend Development 44
3.3.2 Backend Development 46
3.3.3 Database Design 48
3.3.4 WebSocket Communication 49
3.4.1 Room and User Workflow 51
3.4.2 Message Processing 52
3.4.3 Participant Tracking 53
3.4.4 Typing Indicators and User Interface 54
3.6.1 Setup and Execution 57
3.6.2 Limitations and Challenges 59
4.1.1 Report Preparation 60
4.1.2 Project Management and Communication 60
4.1.3 Testing, Characterization and Data Validation 61

LIST OF FIGURES

Fig 1 High-level application architecture 45
Fig 2 React page and component structure 46
Fig 3 REST request and response flow 47
Fig 4 MongoDB document relationship 47
Fig 5 Message broadcast and persistence sequence 48
Fig 6 Room creation and joining workflow 48
Fig 7 Participant session tracking 49
Fig 8 Typing indicator event flow 49
Fig 9 STOMP over SockJS connection flow 50
Fig 10 Chat screen interaction layout 51
Fig 11 Application setup workflow 51
Fig 12 User registration and login workflow 53
Fig 13 Message validation sequence 53
Fig 14 Testing and validation overview 64

CHAPTER 1
INTRODUCTION

1.1. Need Identification

Real-time communication is a routine requirement for study groups, project teams and online communities. In a basic chat arrangement, messages may be exchanged through delayed refreshes or separate channels, which makes conversation difficult to follow. A shared room with live delivery gives several users a common place to communicate and observe participation.

The project addresses this need through PulseChat, a browser-based multi-user chatroom application. The inspected source implements account signup and login, room creation and joining, real-time text messages, stored message history, online participant tracking, typing events and connection status. These capabilities appear together in one interface and are supported by a server that mediates communication.

Project context

A chatroom differs from a one-to-one exchange because each message is associated with a room and made available to all clients subscribed to that room. The application therefore has to coordinate room membership, concurrent sessions, data storage, user interface state and network connectivity. A robust design must also keep stored history separate from transient events such as typing status.

Real-time delivery and data persistence

The implementation uses HTTP endpoints for account and room operations and a STOMP messaging channel for live events. This division suits the different interaction patterns: REST requests return a finite response, while a WebSocket connection remains available for events that may arrive at any time. SockJS is configured as the transport layer for the STOMP endpoint.

A persistent store is needed so that the latest messages remain available after a user leaves and returns to a room. MongoDB is configured at localhost:27017 with the chatApp database. Room documents contain a room identifier and an embedded message list; user records are stored separately. Online participants, by contrast, are tracked in memory because they represent active server sessions rather than durable records.

The need is therefore not limited to sending text. Users also need room discovery by identifier, readable history, clear connection feedback and awareness of other participants. The project brings these interactions into one application and provides a concrete base for further improvements in security, scalability and usability.

1.1 Need Identification: User needs

Need	Implemented response
Synchronous conversation	STOMP message sent to a room destination and broadcast to subscribers
Return to earlier discussion	Room messages are stored in MongoDB and retrieved through a paginated REST endpoint
Shared group context	Room identifiers, participant list and join/leave events
Presence feedback	Participant sessions tracked in concurrent in-memory maps
Conversation cues	Typing events, connection badge, message bubbles and toast notifications
Account access	Signup/login endpoints with BCrypt password hashing

The feature set aims to make basic multi-user communication understandable without implying that rooms are access-controlled private spaces. The interface describes room creation and joining, but the inspected backend does not associate room membership with an authorization rule. Future security work must address that distinction.

1.1 Need Identification: Scope

The current scope is a single web application with a React client, a Spring Boot server and MongoDB persistence. The client supports the landing and chat routes, authentication forms, a room workflow, message history and a responsive chat layout. The server exposes authentication and room REST controllers and a STOMP message controller.

The implemented data types include User, Room and Message. A user has a display name, email, password hash and creation time. A room stores its identifier and embedded messages; each message records a sender, content and local date-time. Participant and typing events are runtime communication data and are not persisted as message history.

The project does not provide evidence of deployment to a public host, end-to-end encryption, a durable session/token mechanism, a moderation console or automated load testing. These are outside the verified scope and should not be represented as completed features.

1.2. Identification of Problem

Users in a group discussion need to see messages as they are sent, while returning participants need access to recent conversation context. A design based only on periodic page refresh does not provide immediate delivery and can create duplicated or stale interface state. A solution must deliver each message to the intended room participants and maintain an ordered history.

The system also faces concurrent activity: multiple browser sessions may join, send messages and disconnect independently. A participant count derived from a single global username list would be inaccurate when users share names or have more than one session. Correct cleanup on disconnect is necessary to avoid stale presence information.

In addition, communication can fail or be delayed. Users need visible connection state and the application must avoid accepting blank or oversized message content. Room identifiers and account input also require validation to reduce malformed requests.

Problem statement and security considerations

The identified problem is to build a web chat system that combines room-based real-time delivery, persistent message retrieval and session-aware presence while presenting a usable interface. The server must distinguish persistent messages from ephemeral typing events, validate messages, store accepted content and broadcast it to room subscribers.

Account operations add another concern: email addresses should be normalized and duplicate signup should be rejected. Passwords must not be stored in plain text. The backend uses BCrypt hashing and checks a submitted password against the stored hash on login. However, the inspected login response does not issue a signed session token, so authentication should not be confused with authorization enforcement on every room operation.

The project's design addresses the core communication flow. It leaves production-grade access control, secure transport configuration and horizontal scaling as areas requiring further engineering and validation.

1.2 Problem statement and expected outcome

Problem statement: How can multiple users communicate in named rooms through a browser application, receive live messages and presence updates, and retrieve recent messages after reconnecting, while keeping the user interface responsive and the server logic organized?

Expected outcome: a working client-server application in which a user can register or log in, create or join an existing room, load the most recent messages, connect through STOMP over SockJS, send and receive room messages, observe typing and participant events, and leave the room. The backend persists accepted messages in MongoDB and exposes room history through REST.

The outcome is derived from implementation files, including ChatController, RoomController, AuthController, RoomParticipantService, WebsocketConfig, the React page components and the STOMP client hook. It is not a claim that every operational scenario has passed formal testing.

1.3. Identification of Tasks

The project work can be divided into requirements analysis, backend setup, data modelling, REST API implementation, WebSocket configuration, frontend workflow development, integration and documentation. Each task corresponds to a visible project component rather than a presumed development chronology.

The backend begins in ChatAppBackendApplication and is built with the Maven wrapper. Spring Data MongoDB supplies repositories; Spring Web handles HTTP requests; Spring WebSocket provides the message broker integration. Lombok generates routine accessors and constructors for entity and data classes.

1.3.1 Requirements and planning

Requirements include account registration and login, room creation and joining, message history, live message delivery, participant visibility, typing feedback and a clear path to leave a room. Technical requirements include validation, cross-origin configuration for the local frontend, MongoDB connection settings and support for concurrent WebSocket sessions.

Frontend and backend development

1.3.2 Interface and frontend development

The frontend is built with React 19 and Vite. React Router maps the root route to LandingPage and /chat/:roomId to ChatPage. Reusable components provide authentication forms, avatars, message bubbles, typing indicators, connection badges and toast notifications. CSS Modules scope styling to pages and components, while global CSS holds shared rules.

The landing workflow validates a display name for unauthenticated guests and a room identifier. Users can create a room or join an existing one. The chat page loads messages and participants, establishes the STOMP client, renders room events and provides a message composer. Enter sends a message and Shift+Enter inserts a new line.

1.3.3 Backend and API development

AuthController handles signup and login. RoomController validates room identifiers, creates rooms, returns room information, retrieves a page of messages and provides a participant list. ChatController handles message, typing, join and leave destinations. WebsocketConfig declares /chat as the SockJS endpoint, /app as the application prefix and /topic and /queue as simple broker prefixes.

Database and account management

1.3.4 Data design and persistence

The Room MongoDB document has a roomId and a list of embedded Message objects. Message records sender, content and time. User documents contain name, unique indexed email, password hash and createdAt. UserRepo and RoomRepo provide repository access. Message history is read from the embedded room list and returned in chronological order within each page.

The application stores room and message data durably in MongoDB. Participant session maps are held in a RoomParticipantService using ConcurrentHashMap-backed collections and are cleared when sessions disconnect. The configured default Mongo URI points to a local instance; deployment configuration is not present in the inspected files.

1.3.5 Authentication and integration

Signup normalizes email to lowercase, validates the display name, email format and a minimum password length, rejects duplicate email and stores a BCrypt hash. Login retrieves by normalized email and checks the password hash. The returned AuthResponse contains identity details, but no JWT or durable login token is generated by the inspected controller.

Real-time events and integration

1.3.6 Real-time events

The STOMP client subscribes to room messages, typing events and participant updates. It sends messages to /app/sendMessage/{roomId}, typing state to /app/typing/{roomId}, and join/leave notices to the corresponding application destinations. The server uses the room path as the authoritative room identifier for message handling.

Participant registration maps a WebSocket session to a room and username. A concurrent set maintains sessions per room. Join and leave events publish the current participant count and distinct user names. A disconnect listener removes sessions that close without an explicit leave.

1.3.7 Integration and refinement

Integration connects the React API modules to REST endpoints and the STOMP hook to the WebSocket endpoint. Message history is merged with live events using a key composed of sender, timestamp and a content prefix to reduce duplicates. The interface tracks loading, error and connection state and offers retry or navigation behavior for failed room loading.

Verification and documentation

1.3.8 Verification and documentation

The repository contains a Spring context-load test with repository and messaging dependencies mocked. It is a narrow startup test, not a comprehensive suite for authentication, room validation, message ordering, concurrent sessions or frontend behavior. No recorded performance measurements or manual test report was included in the project folder.

Documentation tasks include recording prerequisites, configuration, local startup steps, API destinations, data flow, implementation boundaries and future work. A reproducible report should distinguish verified source behavior from recommendations that remain unimplemented.

1.4. Timeline

The source folder does not provide dated commits, a project schedule or a formal work log. The following timeline is therefore an implementation sequence suitable for describing the work, not a claim about calendar dates or measured duration.

Schedule evidence and report structure

Phase	Work products
1. Requirement definition	User needs, functional scope and system boundaries
2. Technology setup	Java 21, Spring Boot, Maven wrapper, React/Vite dependencies and MongoDB configuration
3. Backend foundation	Entities, repositories, account endpoints and room endpoints
4. Real-time layer	SockJS/STOMP endpoint, broker destinations, chat controller and presence tracking
5. Frontend workflow	Landing page, authentication, room flow, chat view and reusable interface components
6. Integration	REST history combined with WebSocket updates; connection and error states
7. Review	Source inspection, context-load test definition and report preparation

A project diary or version-control history would be required to confirm when each phase occurred and how long it took.

1.4 Timeline and 1.5 Report organization

1.5. Organization of the Report

Chapter 1 introduces the need, problem, tasks, timeline and organization. Chapter 2 reviews the communication background, proposed solution, related technology concepts, problem definition, objectives and selection rationale. Chapter 3 describes requirements, design constraints, architecture, data and event flows, implementation plan and setup. Chapter 4 summarizes implemented behavior and validation evidence. Chapter 5 presents conclusions and proposed future work. References identify the official technical documentation consulted for terminology and framework descriptions.

The chapter labels and sequence in this report follow the body structure of the supplied report. The contents page in the PDF described a different chapter outline, so its entries were reconciled to the headings actually present in the report body.

CHAPTER 2
BACKGROUND STUDY

2.1. Timeline of the Reported Problem

Networked messaging evolved from asynchronous message exchange toward persistent connections and event-driven delivery. Modern browser applications can maintain a bidirectional channel through WebSocket, while a messaging protocol such as STOMP provides named destinations and publish-subscribe semantics above the transport.

For a room chat, the central challenge is coordinating several client sessions that share a topic. A server-side broker routes published messages to subscribed clients. Durable history remains a separate concern: a broker delivers live events but does not by itself provide an archive. PulseChat therefore combines a messaging broker with MongoDB-backed room data.

Persistent data and session state

A typical chat service separates durable entities from transient state. Accounts, rooms and messages may survive a restart; typing status and current online presence usually do not. The project reflects this distinction by storing user and room documents while tracking online sessions in memory.

Concurrent participation also requires safe shared state. RoomParticipantService uses concurrent maps and sets so separate WebSocket event handlers can update session membership without relying on a single mutable list. This supports concurrent access within one server process, although it does not synchronize state across multiple server instances.

Browser and server architecture

Modern applications commonly divide responsibilities between a browser client, an HTTP API, a real-time channel and a persistence layer. The client presents user interaction; REST handles finite operations such as creating a room; WebSocket handles ongoing events; MongoDB stores durable records.

This separation helps explain the project architecture and its limits. The local configuration demonstrates development-time integration. It does not establish deployment readiness, reliability under load or operational security in a public environment.

Frontend and backend frameworks

Browser-side frameworks make it practical to express the interface as reusable components and state transitions. React manages updates when messages arrive, when a user types, and when a connection changes. Vite supplies the frontend development and build workflow. CSS Modules isolate presentation styles.

On the server, Spring Boot organizes HTTP controllers, repositories, services and WebSocket configuration. Spring Data MongoDB maps Java objects to documents. The framework structure makes components easier to inspect, but careful validation and test coverage remain necessary.

History retrieval and duplicate handling

A chat client must reconcile the initial history response with later live messages. The project loads recent messages through REST and subscribes to the room topic for new events. The frontend derives a message key from sender, time and content prefix to avoid duplicate insertion when overlapping data appears.

This approach is pragmatic for the current prototype. A stable server-generated message identifier would provide more reliable deduplication, especially when timestamps collide or message text shares the same prefix.

Presence and concurrent sessions

Presence is an approximation of active transport sessions. The backend associates a session id with a room and username, publishes JOIN and LEAVE events, and removes membership on disconnect. A displayed participant count therefore reflects active tracked sessions within the current server process.

It should not be interpreted as a permanent user directory or a guarantee that a browser remains actively attended. Reconnection handling, session expiry and multi-instance coordination require additional design.

2.2. Proposed Solution

The proposed solution is the implemented PulseChat web application: a React client communicates with a Spring Boot backend over REST and STOMP over SockJS. MongoDB stores users, rooms and room message history. The design combines room-based delivery with a single chat interface and separate controls for account and room workflows.

A user can register or log in, choose to create or join a room, and navigate to its chat route. The client requests recent history and participant information, connects to the room topics and sends messages through an application destination. The server validates, stores and broadcasts each accepted message.

Room and history capabilities

2.2.1 Centralized room communication

Each room has a textual identifier. Room creation accepts identifiers from 3 to 50 characters consisting of letters, digits, underscores and hyphens, and returns conflict when the identifier already exists. Joining checks that the room exists. These rules help users share a room id, but do not make a room private or require an invitation.

2.2.2 Stored message history

Messages are appended to the room document and saved through RoomRepo. A REST history request defaults to page zero and size 50; size is clamped to 1-200. Page zero selects the newest records and returns them oldest-first. Older pages are selected by increasing the page number.

Live events and expected benefits

2.2.3 Live room events

STOMP subscriptions use /topic/room/{roomId} for chat messages, /typing for typing state and /participants for participant updates. The client sends to /app/sendMessage/{roomId}, /app/typing/{roomId}, /app/join/{roomId} and /app/leave/{roomId}. This convention separates client requests from broker topics.

Typing events are broadcast without database persistence. Participant events are generated by a server-side service and are likewise ephemeral. Chat messages are durable and appear in history after retrieval.

2.2.4 Expected benefits

The architecture provides immediate room updates, a retrievable recent history, clearer presence feedback and distinct responsibilities across client, controller, service and repository layers. These are design benefits; user satisfaction, throughput and latency have not been measured in the supplied project.

User feedback and operational boundaries

The application handles failed operations with validation messages, toast notifications, room-not-found behavior and a connection badge. The chat interface disables sending while disconnected, supports Enter-to-send and Shift+Enter for multiline content, and includes a scroll-to-bottom affordance when a reader moves up through history.

The project also includes a signup and login form. Email addresses are trimmed and lowercased, duplicate emails are rejected, and passwords are hashed with BCrypt before persistence. However, there is no token issuance or room-level access check in the inspected implementation; the user interface login state alone should not be considered strong authorization.

Local configuration and deployment scope

The solution is intended for local development and demonstration. The application properties point to a local MongoDB instance, the backend uses port 8081 and the Vite development server conventionally uses port 5173. CORS and WebSocket origins include local development addresses.

For deployment, environment-specific configuration, HTTPS/WSS, secure session handling, authorization, rate limits, monitoring, backup and a production broker/database strategy would need to be designed and verified. These are proposed improvements rather than current capabilities.

2.3. Bibliometric Analysis

A formal bibliometric study requires a documented search strategy, databases, date range, inclusion criteria and analyzed publication set. None was supplied with the project source. This section therefore provides a qualitative technology and literature context, not publication counts or quantitative citation statistics.

Research and engineering writing on real-time web applications commonly discusses persistent connections, publish-subscribe messaging, concurrent state, durable storage and user experience. Those themes map directly to the design choices in this project.

WebSocket and STOMP messaging

WebSocket is a standardized protocol for bidirectional communication between a client and server over a single connection. STOMP supplies a frame and destination model that allows clients to subscribe and publish messages. SockJS provides a client/server library intended to support WebSocket communication with fallback transports where needed.

In this project, Spring's simple message broker routes topic messages within the application process. The configuration uses /topic and /queue broker prefixes, /app for application destinations and /user for user destinations. No external message broker is configured in the inspected files.

Publish-subscribe design

Publish-subscribe messaging is useful when one event should reach several interested clients. A room topic represents that shared audience. The server maps the room-specific application destination to its room topic, persists the message, and returns the message for broadcast.

This pattern is distinct from a request-response API. An HTTP request completes with a response; a topic subscription remains active for future events. Keeping the two flows separate clarifies message history versus live delivery.

Document storage model

Document databases organize records as flexible documents. The project stores each room and its messages in MongoDB, embedding messages within the room document. This simplifies loading and saving a room's history for a small application, while large unbounded message arrays may become a scaling concern.

A future design could store each message as a separate document with a room index and timestamp, enabling more scalable pagination and retention policies. The present implementation uses the embedded list and should be described accordingly.

Component-based interface design

Component-based interface design encourages reuse and predictable rendering. The application has dedicated components for messages, avatars, connection state, typing status and authentication. The chat page coordinates data loading and real-time subscriptions, while the API modules isolate HTTP calls.

These choices reduce duplication and help locate responsibilities. The inspected frontend uses JavaScript JSX; TypeScript appears in the TypeScript configuration and package dependency, but the application source files shown are JSX rather than TypeScript.

Security considerations

Security guidance for web applications emphasizes password hashing, input validation, transport protection and authorization checks. The project implements BCrypt password hashing and validation for selected inputs. It does not demonstrate end-to-end encryption, HTTPS deployment, role-based access or authenticated WebSocket authorization.

The project's account API is a useful foundation, but production use requires a security review. In particular, login success is not evidence that later room and message operations are protected against impersonation.

Research gap and contribution

For the chatroom context, the practical gap addressed by this project is integration: live delivery, stored recent history and participant awareness are assembled in one browser workflow. The contribution is an implemented prototype that demonstrates these concepts with a Java/Spring backend and a React frontend.

The project does not claim novel messaging protocols or publish-subscribe algorithms. Its academic value is in requirements analysis, system integration, implementation and reflection on limitations.

2.4. Review Summary

The background review establishes the distinction among request-response communication, real-time event delivery and persistent storage. PulseChat uses REST for account and room actions, STOMP over SockJS for ongoing room events, and MongoDB for durable account, room and message records.

The system also tracks active sessions in concurrent in-memory structures and broadcasts participant changes. This makes presence responsive within one process while making it transient across a server restart. The design is suitable for a learning prototype and exposes clear opportunities for stronger security, scalable persistence and automated evaluation.

2.5 Problem Definition

The problem is to implement a multi-user web chat application where multiple independent clients can join named rooms, exchange text messages in real time, review recent history and receive participant and typing feedback. The application must validate user input, persist messages, indicate connection status and clean up participant state on disconnect.

The problem includes user workflow as well as server correctness. A room identifier must be validated and checked for conflicts; a message must have a sender and nonblank content within the configured maximum length; a room must exist before a message can be stored and broadcast. Concurrent clients must subscribe to the correct room destination.

Scope and implementation boundary

The scope is bounded by the project code: browser-based client, Spring Boot backend, local MongoDB configuration, REST endpoints, SockJS/STOMP messaging, and a single-process in-memory participant service. Authentication endpoints validate credentials and hash stored passwords, but the implementation does not create a signed token or enforce authenticated identity on chat messages.

The system is evaluated from source evidence and the repository's test definition. No claims are made about measured latency, capacity, security certification or live deployment. This boundary keeps the report accurate and identifies meaningful future work.

2.6 Goals/Objectives

• Provide a browser interface for users to register or log in.
• Allow users to create rooms and join existing rooms by identifier.
• Deliver text messages to clients subscribed to the same room.
• Persist room messages in MongoDB and expose paginated history.
• Track connected WebSocket sessions and publish participant updates.
• Broadcast typing state as a transient room event.
• Expose connection, loading and error state in the user interface.
• Organize code into frontend components and backend controller/service/repository layers.

2.6 Goals and measurable scope

Objective area	Evidence in source
Account handling	AuthController signup/login; BCryptPasswordEncoder; unique email index
Room lifecycle	RoomController create, lookup and participant endpoints
Message delivery	ChatController @MessageMapping and @SendTo
Persistence	Spring Data MongoDB repositories and Room document
Presence	RoomParticipantService and disconnect event listener
Interface	LandingPage, ChatPage, connection and message components
Verification	One Spring context-load test; no broader recorded test results

These objectives describe implemented or inspectable behaviors. Items such as invitation privacy, authenticated room membership, message editing and mobile-native support are not included in this list because they are not implemented.

2.7 Why Use These Technologies Over Others

React was selected for a component-driven interface with state updates when network events arrive. Vite provides a development server and frontend build pipeline. The source currently uses React JSX and CSS Modules, which offer scoped component styling without a utility CSS framework.

Spring Boot brings HTTP and WebSocket support in a single Java application. Spring Data MongoDB maps entities and repositories to the document store. Maven Wrapper supplies a project-specific build entry point without requiring a separate Maven installation.

Technology comparison criteria

Technology	Role in this project	Reason in context
Java 21	Backend language	Typed server-side code and Spring ecosystem
Spring Boot 3.4.2	Application framework	Organizes REST, WebSocket, configuration and dependency wiring
Spring Web	REST API	Handles account and room HTTP endpoints
Spring WebSocket	Messaging	STOMP endpoint and simple broker integration
MongoDB	Persistence	Stores user and room documents with embedded message history
React 19 / Vite	Frontend	Component-based browser interface and dev/build workflow

STOMP over SockJS

STOMP over SockJS is used for the room's event channel. STOMP names destinations and supports subscriptions, while SockJS is configured at /chat. This is more expressive for room subscriptions than manually encoding all event routing in raw socket frames.

A simple broker is enabled in the Spring configuration. It is an in-process broker suitable for development and modest demonstrations. An external broker would be considered for larger deployments or coordinated multi-instance delivery.

MongoDB document storage

MongoDB is used rather than a relational database in the current implementation. Room records embed message lists, and User documents store account fields. The schema is straightforward for a prototype, but an unbounded embedded history may need redesign when rooms accumulate many messages.

The configuration uses a local MongoDB URI and the database name chatApp. There is no evidence of a hosted database service, migrations, or a formal backup process.

BCrypt password hashing

BCrypt is used for password hashing through Spring Security Crypto. It is a password-hashing function, not encryption that can be reversed to recover the original password. The application checks the submitted password with the stored hash during login.

Authentication is not complete access control. The source does not issue JWTs or apply a security filter to require a logged-in principal for room APIs or STOMP messages. The report therefore distinguishes password hashing from authorization.

Frontend libraries and styling

The frontend uses React Router for page navigation, @stomp/stompjs and sockjs-client for messaging, and lucide-react for icons. Vite and React are declared in package.json. CSS Modules and global CSS are used for presentation; Tailwind CSS and TypeScript are not the primary implementation approach evident in the source files.

This selection matches the current code and avoids attributing packages or frameworks that are not declared or used.

Build tools and testing

Lombok reduces routine Java boilerplate for data classes and constructor injection. The backend is built with Maven Wrapper and includes Spring Boot Test dependencies. A small context-load test is present, though the project's quality assurance needs broader unit and integration coverage. Technology choices are assessed here in relation to the implementation. No comparative benchmark was conducted, so the report does not assert that these technologies are universally faster or superior to alternatives.

CHAPTER 3
DESIGN AND PROCESS

3.1 Evaluation and Selection of Specifications/Features

The selected features follow the main user journey: account access, room creation or joining, chat connection, message exchange, history retrieval and room exit. Supporting status feedback helps users understand whether the real-time channel is ready. Participant and typing updates provide additional context without being stored as conversation messages.

Functional specifications

3.1.1 Functional specifications

ID	Requirement	Implemented behavior
F1	Register and log in	Signup/login REST routes with input validation and BCrypt
F2	Create room	POST room endpoint validates identifier and detects duplicates
F3	Join room	GET room lookup returns not found for unknown room
F4	Send message	STOMP handler validates, persists and broadcasts
F5	Read recent history	Paginated REST endpoint returns chronological page
F6	Show presence	Session-aware participant events and list endpoint
F7	Show typing	Transient typing event broadcast by room
F8	Show connection state	Frontend connection badge and guarded composer

Non-functional specifications

3.1.2 Non-functional specifications

Usability requires clear room workflow, readable message bubbles, visible connection state, responsive layout and nonblocking feedback. Maintainability is supported by separated API modules, reusable React components and backend controller/service/repository layers. Concurrent access is considered in participant tracking through concurrent maps and sets.

Reliability depends on MongoDB availability for account, room and message operations and on the WebSocket connection for live messaging. Scalability is limited by the in-process simple broker, in-memory presence state and embedded message arrays. Security is partial: password hashing and selected validation exist, but production authorization and secure transport are not demonstrated.

Technology selection

3.1.3 Evaluation and selection of technologies

The project uses Java 21 and Spring Boot 3.4.2 in the backend. The dependencies include Spring Web, Spring WebSocket, Spring Data MongoDB, Spring Security Crypto and Lombok. The frontend package declares React 19, Vite, React Router, STOMP.js, SockJS client and Lucide React.

The repository’s package.json and pom.xml are the authoritative version sources. The Vite config, source imports and application properties establish the development setup. The implementation is JavaScript JSX; TypeScript is present as a package dependency/configuration but no TypeScript application source was found in the inspected file inventory.

Core feature selection

3.1.4 Selection of core features

The core feature set prioritizes communication and room context: sending messages, receiving broadcasts, retrieving history, tracking participants and showing connection state. Account flows provide a basic identity record. The design keeps typing and presence ephemeral because they describe current activity rather than durable conversation content.

The frontend also supports message deduplication, automatic scroll behavior, an explicit jump-to-bottom button, toast messages and responsive sidebar behavior. These are usability details supported in ChatPage and component files.

Data management and security

3.1.5 Data management and security considerations

User email has a unique index. Signup converts email to lowercase, validates a regular-expression pattern, checks duplicates, hashes the password and returns a response that omits the password. The Room entity stores embedded Message records. Message content is trimmed and limited to 2000 characters before persistence.

The application does not demonstrate database encryption, end-to-end encryption, JWT, durable session tokens, role-based permissions, or authenticated WebSocket principals. Room ids are identifiers, not access credentials. These constraints should be addressed before handling sensitive or public traffic.

Final evaluation

3.1.6 Final evaluation

The selected design satisfies the prototype's central objective: room-scoped live chat with persistent recent history and presence feedback. It uses a modest set of technologies that map directly to the required interactions.

The design is not a production deployment blueprint. Message volume, throughput, latency, memory behavior and security posture have not been empirically measured. The test source checks Spring context wiring only.

3.2 Design Constraints

Design constraints arise from the local configuration, framework choices, single-process broker, embedded data model and absence of a deployment environment. The system also depends on a running MongoDB instance and browser access to the configured backend and frontend origins.

Technology and environment

3.2.1 Technology and environment constraints

The backend requires Java 21 as configured in the Maven project. The frontend uses Node.js and npm for the Vite workflow. The Maven wrapper is included. The application properties configure the backend on port 8081 and connect MongoDB at localhost:27017/chatApp.

WebSocket and CORS configuration explicitly allow selected localhost origins. A different host or port requires configuration changes. No environment variable-based production profile was identified in the inspected files.

Database and persistence

3.2.2 Database and persistence constraints

A room document contains an embedded list of messages. This makes a room's data easy to load as one aggregate, but message arrays can grow indefinitely. The history endpoint slices the in-memory list and caps page size at 200; it does not use database-side message pagination.

MongoDB is required for account, room and message operations. If it is unavailable, durable operations cannot complete. The local URI and lack of backup/retention configuration constrain this project to development-scale use.

Real-time communication

3.2.3 Real-time communication constraints

The simple message broker and participant maps are local to one backend process. If traffic is distributed across several instances, subscribers and presence records will not automatically be shared. A broker relay or another distributed messaging approach and coordinated session state would be required.

Network disconnects can interrupt live messaging. The disconnect event listener removes tracked participants, but reconnect synchronization and duplicate-session behavior need broader testing. A live message is received only while the client has an active subscription; persistence supports later history retrieval.

Security

3.2.4 Security constraints

Passwords are BCrypt-hashed and input checks exist for account fields, room ids and message content. However, the backend does not establish authenticated identities for STOMP messages or protect room endpoints with a security policy. A sender name is included in the message request and should not be trusted as proof of identity.

Local HTTP endpoints do not demonstrate TLS. Before deployment, the application needs HTTPS/WSS, authorization, abuse controls, secret management, error handling review and security testing.

Performance and usability

3.2.5 Performance and usability constraints

The application has no recorded load or latency measurements. Embedded message history may increase document size and serialization cost. In-memory participant tracking consumes resources proportional to active sessions. The UI has auto-scroll behavior and a scroll-to-bottom control, but very long histories may require virtualization or incremental loading.

These are anticipated constraints based on the design, not observed benchmark outcomes.

Development and maintainability

3.2.6 Development time and maintainability

The repository includes a modular structure but no formal schedule, requirements traceability file or release history in the inspected folder. Maintaining alignment across REST paths, WebSocket destinations, DTOs and frontend hooks requires coordinated changes.

An API contract, stronger automated tests and consistent configuration profiles would improve maintainability. The current context test does not validate business rules or client behavior.

Overall design considerations

3.2.7 Overall design considerations

The design favors understandable separation of roles: React components manage interaction, API modules call REST endpoints, Spring controllers handle requests, a participant service coordinates session state, repositories access MongoDB and the simple broker distributes events.

Future changes should keep message persistence, authorization and event routing explicit. Growth beyond a single-process demonstration will require revisiting storage, broker and participant coordination.

3.3 Design Flow

The architecture has four principal parts: the browser interface, REST controllers, the STOMP message channel and MongoDB. The user interacts with the browser, which calls REST endpoints for account, room and history operations and uses a persistent SockJS/STOMP connection for live events.

React browser → **REST + STOMP** → **Spring Boot** → **MongoDB**

Fig 1. High-level application architecture

3.3 Design Flow: frontend development

The React application mounts in main.jsx and renders App. BrowserRouter declares the landing route and room chat route. ToastProvider and AuthProvider wrap the routes. LandingPage manages account presentation and create/join selection; ChatPage manages history, participants, messages, typing state and connection status.

The frontend API modules encapsulate HTTP calls. The STOMP hook handles connection lifecycle, subscriptions and publish operations. MessageBubble, Avatar, TypingIndicator and ConnectionBadge render focused interface elements. CSS Modules scope page and component styles.



Fig 2. React page and component structure

3.3 Design Flow: frontend modules

Module	Responsibility
App.jsx	Routes and context providers
LandingPage.jsx	Account-aware landing screen and room actions
ChatPage.jsx	History load, event handling, composition and display
roomApi.js	Room REST requests
authApi.js / AuthContext.jsx	Authentication requests and client auth state
useStompClient.js	STOMP/SockJS connection and room subscriptions
useTypingIndicator.js	Typing start/stop timing
MessageBubble / ConnectionBadge	Reusable message and status rendering

3.3 Design Flow: backend modules

AuthController implements signup and login under /api/v1/auth. RoomController implements creation, room lookup, message history and participant listing under /api/v1/rooms. ChatController handles STOMP destinations for sending messages, typing, joining and leaving.

RoomParticipantService tracks session-to-room and session-to-user mappings and room-to-session sets. WebSocketEventListener cleans up a session on disconnect. WebsocketConfig registers the /chat SockJS endpoint, configures allowed local origins, sets destination prefixes and enables an in-process simple broker. Repositories persist Room and User entities.

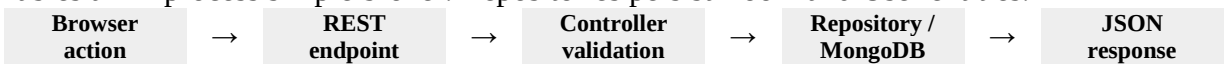


Fig 3. REST request and response flow

3.3 Design Flow: database design

Document	Fields / contents	Persistence role
User	id, name, email, password hash, createdAt	Account record; email uniquely indexed
Room	id, roomId, embedded messages[]	Room record and chat history
Message	sender, content, time	Embedded in the room document
Participant state	session-room/user maps and room session sets	In-memory only; cleared on disconnect

This model distinguishes stored conversation data from live presence. The participant maps do not survive a process restart. The room identifier is validated by RoomController and used by the message handler to find the room.



Fig 4. MongoDB document relationship

3.3 Design Flow: message transmission

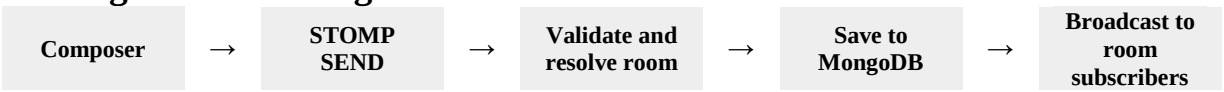


Fig 5. Message broadcast and persistence sequence

The client trims input and prevents blank sends. ChatController checks for nonblank content, enforces a 2000-character limit, requires a sender and verifies that the target room exists. It creates a timestamped Message, appends it to the room and saves the room before returning the message for broadcast.

The destination room id is taken from the STOMP path variable rather than trusting a potentially inconsistent room id in the payload. This is an input consistency measure; it does not authenticate the claimed sender.

3.3 Design Flow: room workflow

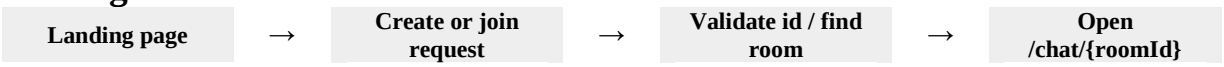


Fig 6. Room creation and joining workflow

Room creation accepts JSON or plain text input and extracts the room identifier. It requires 3-50 characters and permits letters, numbers, hyphens and underscores. Duplicate identifiers return a conflict response. Joining retrieves an existing room or returns not found.

The client may generate a suggested identifier from a fixed adjective/noun list and a three-digit number. The generated value is a convenience, not a cryptographic random token or room access secret.

3.3 Design Flow: participant tracking

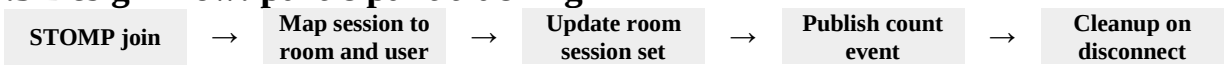


Fig 7. Participant session tracking

On join, the server registers the WebSocket session id, room id and username, then broadcasts a participant event. On explicit leave or transport disconnect, it removes the mapping and publishes an updated event. getParticipants returns distinct sorted usernames, while the event count is based on active sessions.

If a user opens multiple sessions, the count may exceed the number of distinct names. The UI currently uses participant list length or the event count depending on the display location.

3.3 Design Flow: transient typing events

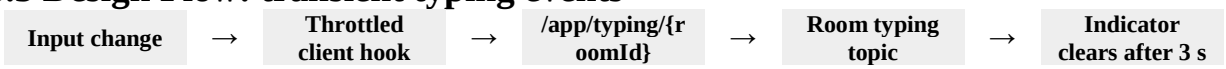


Fig 8. Typing indicator event flow

Typing events are not persisted. The frontend suppresses its own typing state, tracks active senders and clears stale indicators after three seconds. The typing hook throttles event emission to avoid sending on every keystroke. This event path is separate from chat message history.

3.3 Design Flow: history and duplicate handling

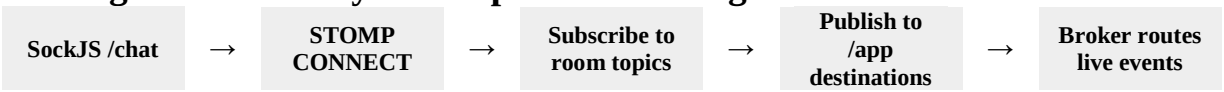


Fig 9. STOMP over SockJS connection flow

When a room loads, the client requests the first history page and the current participant list. The returned messages initialize the visible state and a set of seen keys. Incoming live messages are checked against that set and against current message state before append.

The key combines sender, time and the first 40 characters of content. A server-side unique message id would be a stronger long-term identity strategy.

3.3 Design Flow: data and service boundaries

MongoDB stores user documents and room documents with embedded messages. The simple broker routes events for active subscribers. Participant tracking resides in concurrent in-memory collections; typing event payloads pass through the broker and are discarded after delivery.

The separation clarifies restart behavior: chat history remains in MongoDB while live participant and typing state resets when the backend process stops.

3.3 Design Flow: interface

The landing screen presents PulseChat identity and account access, then supports room creation or joining. In the chat screen, a sidebar displays the room identifier, current user, online participants, connection status and a leave action. The main area shows room messages, typing cues, connection state and a composer.

The UI supports a collapsible sidebar for smaller screens, toast notices for network and participant events, color-coded initials avatars, auto-scroll while near the bottom, and a jump button after scrolling upward. These descriptions are based on the inspected JSX and CSS modules.

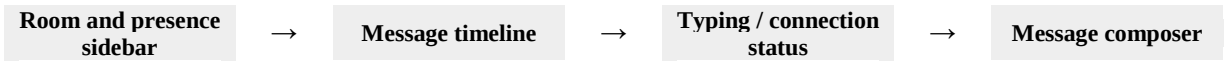


Fig 10. Chat screen interaction layout

3.3 Design Flow: setup workflow



Fig 11. Application setup workflow

The project README requires Java 21, Node.js, npm and MongoDB. The backend can be launched with the included Maven wrapper from chat-app-backend. The frontend is installed and launched with npm from chat-app-frontend. The frontend API and WebSocket URLs can be configured with VITE_API_BASE_URL and VITE_WS_URL as documented in the README.

The committed application.properties value sets server.port=8081 and the local Mongo URI. Configuration should be checked against the running service rather than relying on stale documentation.

3.4 Creation of the Chatroom Application

The application is assembled from a browser client, a Spring Boot server and MongoDB. The backend entry point enables Spring Boot component discovery. Configuration classes enable cross-origin REST requests, register the SockJS endpoint and manage WebSocket lifecycle events.

The frontend is delivered as a Vite project. App.jsx defines the route structure and wraps it in authentication and toast contexts. Page-level components coordinate user actions and shared components display local interaction state.

Account and room setup

3.4.1 Account and room setup

Signup validates a display name between 2 and 50 characters, a basic email pattern and a password of at least six characters. Email is trimmed and lowercased; duplicate email yields HTTP 409. A BCrypt hash is saved to the User document. Login returns unauthorized for an unknown email or mismatched password.

Room creation validates identifier length and allowed characters and returns conflict if the room exists. A room lookup returns the room or a not-found response. The frontend routes a successful room action to /chat/{roomId}.

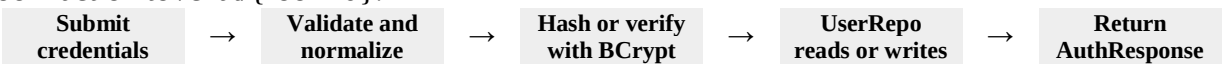


Fig 12. User registration and login workflow

Message handling

3.4.2 Message handling

A message request contains sender and content. The server rejects blank content, content over 2000 characters, a missing sender and a nonexistent room. The accepted content and sender are trimmed, the timestamp is assigned by the server, and the message is appended to the room before saving.

The STOMP method returns a Message and uses @SendTo to broadcast to /topic/room/{roomId}. History is exposed by a REST route that returns a slice in chronological order. Page size is clamped from 1 through 200.

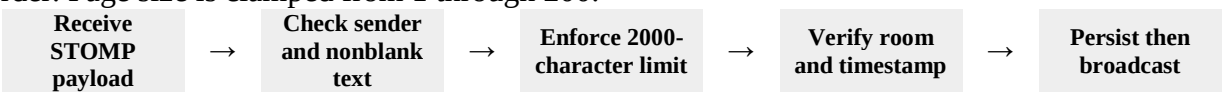


Fig 13. Message validation sequence

Presence and typing

3.4.3 Presence and typing

A join destination registers the current STOMP session. A leave destination removes it. The session disconnect listener also removes stale participant state when the transport ends. Participant events include room id, count, event type, username and user list.

Typing requests overwrite the event room id with the destination variable and are broadcast to the room typing topic. No database write occurs for typing. The client clears a typing badge after three seconds if a stop event is missed.

Interface behavior

3.4.4 Interface behavior

The composer is disabled while disconnected or while a send operation is pending. Empty content is ignored. Enter sends; Shift+Enter inserts a newline. A message input limit is set at 2000 characters and the UI shows a character count near the limit.

Chat history loading and participant loading begin together. A room-not-found error returns the user to the landing page. On message arrival the page updates its local list and, when the user is already near the bottom, scrolls to the newest content.

Important algorithms

3.4.5 Important algorithms

History paging computes the end index as total messages minus page times page size, then selects the preceding page-sized range. This makes page zero the newest page and higher page indices older pages, with chronological order preserved within each result.

Presence updates use a session-to-room map, a session-to-user map and a room-to-session set. ConcurrentHashMap and concurrent key sets support simultaneous access. On removal, the session is deleted from each map and the room set is discarded when empty.

Message deduplication

3.4.6 Message deduplication

The frontend builds a message key from sender, timestamp and a content prefix. On initial load the keys populate a set. On live delivery, the handler first checks the set and then checks the current message array before appending. This protects the visible list from some overlaps between REST history and WebSocket events.

The method has a collision risk because two messages may share sender, timestamp and initial text. A unique message id generated by the backend is recommended for future refinement.

Error and connection handling

3.4.7 Error and connection handling

The frontend shows connecting/connected/disconnected status and guards the send path when the channel is not connected. API failures produce toast messages and inline room or history errors. The landing page handles common conflict and not-found responses with contextual feedback.

The inspected sources do not define a full centralized API error schema or end-to-end retry policy. Network reconnection behavior should be validated under disconnection and server restart conditions.

Implementation boundary

3.4.8 Implementation boundary

Implemented: signup/login endpoints with BCrypt, room creation and lookup, MongoDB persistence, message history pagination, STOMP room broadcasts, typing events, participant session tracking, disconnect cleanup and responsive chat interaction.

Not evidenced as implemented: JWT issuance, room authorization, message encryption, message editing/deletion, moderation, external broker, production deployment, automated load tests or a complete frontend test suite.

3.5 Logo Designing and Naming

The user-facing application is branded “PulseChat” in the React landing and chat interfaces. The name suggests conversational activity and is accompanied by a lightning icon from Lucide React. This branding is an interface label; the academic project title remains “Multi-user Chatroom Application.”

The inspected source includes decorative hero content and a chat preview, but no separate logo design source file or documented design process was supplied. The report therefore describes the visible application identity without claiming a formal branding study.

Visual identity in the interface

The interface uses a lightning symbol, room hash markers, initials-based avatars and status indicators to communicate chat concepts. The Avatar component derives initials and deterministic colors from a username. Such presentation choices are helpful cues, but they do not provide identity verification or security.

The report uses PulseChat only to refer to the application name visible in the source. The project title and student details on the cover remain those provided for this report.

3.6 Implementation Plan/Methodology

The implementation method follows an iterative integration sequence. First define account, room and message behavior; then configure data entities and REST endpoints; then configure the real-time messaging broker; then build the client workflow and connect each page to the backend. Finally review error states and document the verified behavior.

This sequence describes a reasonable reading of the system dependencies. The repository does not include dated planning records, so it should not be read as an exact historical schedule.

Setup and execution

3.6.1 Setup and execution

Component	Setup requirement	Local command / address
Database	MongoDB 6+	Start local MongoDB on port 27017
Backend	JDK 21; Maven wrapper included	mvnw spring-boot:run; port 8081
Frontend	Node.js 18+ and npm	npm install; npm run dev; port 5173
Browser	Modern browser	Open http://localhost:5173

The README describes VITE_API_BASE_URL and VITE_WS_URL settings. Backend application.properties sets the MongoDB URI and server port. Ensure that the browser origin is allowed by both CORS and WebSocket configuration.

Validation approach

3.6.2 Validation approach

A practical verification plan should cover account validation, duplicate email, invalid room ids, duplicate rooms, missing rooms, message length limits, empty messages, history page boundaries, join/leave events, disconnect cleanup and typing timeout behavior. The supplied repository only defines a Spring context-load test and does not record outcomes for this broader plan.

For academic accuracy, these cases are proposed validation checks. They must not be described as executed test results without a test log or observed evidence.

Limitations and challenges

3.6.3 Limitations and challenges

The principal engineering challenges are coordinating REST and WebSocket state, ensuring that message history and live delivery do not create duplicates, keeping presence consistent across disconnects, and handling concurrent sessions safely. The interface must remain understandable when the network is unavailable.

Limitations include process-local broker and presence state, embedded message arrays, lack of strong room authorization, no token-based session implementation, and limited automated testing evidence. These are concrete constraints derived from the implementation.

CHAPTER 4
RESULT ANALYSIS AND VALIDATION

4.1 Implementation of Solution

The implemented system is a browser-based chat application composed of a React frontend, a Spring Boot backend and MongoDB. Its user workflow covers account access, room creation or joining, message history, live message delivery, presence feedback and room exit. REST and STOMP are used for distinct interaction types.

Report preparation and project evidence

4.1.1 Report preparation

The report content is grounded in the repository's source files, package declarations, Maven dependencies, application properties and README. Technical claims are limited to behavior visible in those sources. The document does not claim screenshots, performance results or a completed external deployment that were not supplied.

The report follows the five-chapter body structure of the source report and retains its preliminary certificate, contents, figure list, references and ending placeholder page. The contents entries have been aligned with the body headings.

4.1.2 Project management and communication

The system itself supports real-time communication through room topics and participant events. The repository does not include a project management schedule or team communication record; therefore no team roles, meeting cadence or completion dates are asserted.

Testing and data validation

4.1.3 Testing, characterization, interpretation and data validation

The test source contains one Spring Boot context-load test. It mocks the room repository, user repository and messaging template, and uses a test database URI. This test is intended to check that the application context can wire its components without a live MongoDB or broker dependency.

No test execution result, browser test record, load test, latency result or data-integrity measurement was included. Accordingly, observed results are not invented. The implementation behavior can be described from source, while formal validation remains incomplete.

4.1 Implementation results

Area	Implemented behavior	Evidence boundary
Account	Signup/login; BCrypt hash; duplicate email conflict	No JWT or authorization enforcement identified
Rooms	Validated create and lookup by room id	Room id does not prove access rights
Messages	Validate, timestamp, persist, broadcast	No benchmark or delivery guarantee measured
History	Recent-page selection, chronological within page	Stored as embedded room list
Presence	Session mappings and disconnect cleanup	In-memory and single-process
Typing	Transient room broadcast	Not stored
UI	Responsive chat workflow and status feedback	No user study supplied

Message delivery and persistence

Message handling demonstrates the central integration path: the client sends a STOMP payload, the server validates sender and content, resolves the room, creates a timestamped message, saves the room and broadcasts the result. The client appends unique messages to its local view.

History loading demonstrates the persistence path: REST retrieves a bounded page from the room's message list and the client initializes its display before processing live events. This combination supports continuity across visits, subject to the configured storage being available.

Presence and real-time behavior

Presence handling demonstrates how WebSocket session lifecycle can be reflected in the interface. Join and leave operations update session maps and publish participant data. A disconnect listener removes an abandoned session. Typing events provide a short-lived activity cue without affecting stored history.

The result is a coherent prototype-level feature set. Reliability under network partitions, multiple backend instances and large message collections has not been established by the supplied testing evidence.

4.2 Validation

Validation should check behavior across the client, server, broker and database. The repository contains source validation branches and a context-load test definition, but no completed test matrix. The checks below are proposed; no unrecorded pass results are claimed.

Functional validation

4.2.1 Functional validation

Check account validation, duplicate email, room id constraints, duplicate and missing rooms, blank or oversized messages, persisted history, typing delivery and participant cleanup. These cases are represented in source behavior but were not accompanied by recorded execution results.

Frontend and backend validation

4.2.2 Frontend and backend validation

Exercise route navigation, loading and error states, disconnected send prevention, keyboard behavior, sidebar response, scroll management, HTTP results, STOMP destinations and payload validation. The supplied context test does not cover these cases.

Database validation

4.2.3 Database validation

Confirm unique emails, hashed passwords, room lookup, message persistence and newest-page selection with chronological ordering. The controller clamps history page size to 1-200; no database-backed test results were supplied.

Real-time and security validation

4.2.4 Real-time and security validation

Connect multiple sessions and check room isolation, message delivery, join/leave counts and typing cleanup. Security review must cover authenticated sender identity, room authorization and TLS; the current code does not demonstrate these protections.

Performance validation

4.2.5 Performance validation

Measure connection setup, message latency, throughput at stated session counts, database response time, memory use and history retrieval under a documented workload. No performance measurements were supplied.

4.2.6 Validation summary

The source and context-load test are documented; broader functional, integration, security and performance results remain to be collected.

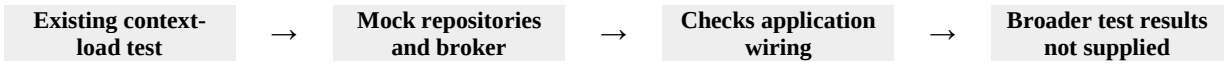


Fig 14. Testing and validation overview

CHAPTER 5
CONCLUSION AND FUTURE WORK

5.1. Conclusion

The Multi-user Chatroom Application demonstrates a room-based communication workflow using a React browser client, a Spring Boot backend, STOMP over SockJS and MongoDB. Users can register or log in, create or join rooms, load recent messages, send real-time messages and see participant and typing updates.

The implementation separates REST operations from ongoing messaging and separates durable records from transient session state. Message content is validated, timestamped, stored with the room and broadcast to subscribers. Participant sessions are tracked concurrently in memory and removed on disconnect. The interface presents connection status and handles common loading and input states.

Project contribution and implementation limits

The project provides a useful foundation for demonstrating full-stack integration and event-driven application design. Its current boundaries are important: the repository does not show token issuance, room authorization, distributed broker configuration, production deployment, end-to-end encryption or comprehensive automated test results.

The application should therefore be understood as a prototype, not a production-secure messaging service. Further engineering should prioritize authenticated identity, access control, reliable persistence scaling, automated tests and deployment configuration.

5.2. Future Work

Future improvements should strengthen identity and access control, scale message storage and broker delivery, and expand automated validation. Each enhancement below is proposed work; it is not represented as already implemented.

5.2.1. Secure authentication and room authorization

Issue a signed session or token after login, validate the authenticated principal on REST and STOMP requests, and enforce room membership before reading history or publishing messages. Add expiry, revocation, CSRF/origin protections as applicable, and TLS for deployed environments.

Scalable message storage

5.2.2. Scalable message persistence

Move messages to a dedicated collection with room and timestamp indexes, stable message identifiers and database-side pagination. Define retention and archival rules to prevent unbounded room documents and preserve efficient history retrieval.

5.2.3. Distributed messaging and presence

Replace the in-process simple broker with a broker relay or distributed messaging service if multiple backend instances are required. Coordinate presence across instances and define reconnect, duplicate session and heartbeat expiry behavior.

Message delivery and moderation

5.2.4. Stronger message identity and delivery

Assign each message a unique server-generated identifier and consider acknowledgements, idempotency keys, ordering policy and client retry behavior. This would improve deduplication and make delivery status more precise during reconnection.

5.2.5. Moderation and room controls

Add room ownership, invitation or access policy, participant removal, reporting and moderation tools. These features require authenticated identity and a clear policy model before they can be safely implemented.

Automated testing and operations

5.2.6. Expanded automated tests

Add unit tests for input validation and paging; controller tests for HTTP responses; repository integration tests; STOMP integration tests with multiple sessions; and frontend tests for key user workflows. Record the environment and actual results for each test.

5.2.7. Observability and operations

Add structured logs, health checks, metrics and alerts for connection failures, database errors and message processing. Provide environment-based configuration, backup/restore instructions, dependency update practices and deployment documentation.

Accessibility and rich interactions

5.2.8. User experience and accessibility

Conduct usability testing and improve keyboard navigation, focus handling, screen-reader labels, contrast and responsive behavior. Consider virtualized rendering for large message histories and a clearer retry experience after disconnection.

5.2.9. Rich communication features

Potential additions include message editing/deletion, attachments, reactions, search and notifications. Each would require explicit persistence, validation, authorization, storage and moderation decisions.

Privacy and data retention

5.2.10. Privacy and retention

Document what account and message data is stored, for how long and who can access it. Define deletion and export processes, secure backups, encryption at rest where appropriate, and a privacy notice aligned with the deployment context.

5.2.11. Deployment and scaling

Prepare separate development and production profiles, container or managed-service deployment, secure secret storage, TLS termination, database availability planning and load testing. Publish performance results only after controlled measurement.

Deployment and next steps

With these improvements, the application could grow from a learning prototype into a more robust collaborative messaging service. The next development phase should begin with identity and authorization, because later moderation and room privacy depend on trustworthy user identity.

Subsequent work should add tests alongside each feature and document acceptance criteria before claiming completion. Scaling decisions should be guided by measured usage rather than assumed requirements.

REFERENCES

1. Spring. Spring Boot Reference Documentation. https://docs.spring.io/spring-boot/index.html
2. Spring. Spring Framework WebSocket Support. https://docs.spring.io/spring-framework/reference/web/websocket.html
3. Spring. Spring Data MongoDB Reference. https://docs.spring.io/spring-data/mongodb/reference/
4. Spring. Spring Security Crypto Module. https://docs.spring.io/spring-security/reference/features/integrations/cryptography.html
5. React. React Documentation. https://react.dev/learn
6. Vite. Vite Guide. https://vite.dev/guide/
7. React Router. Documentation. https://reactrouter.com/
8. STOMP.js. STOMP over WebSocket client documentation. https://stomp-js.github.io/
9. SockJS. SockJS client documentation. https://github.com/sockjs/sockjs-client
10. MongoDB. MongoDB Manual. https://www.mongodb.com/docs/manual/
11. Oracle. Java Documentation. https://docs.oracle.com/en/java/
12. Apache Maven. Maven Wrapper documentation. https://maven.apache.org/wrapper/
13. MDN Web Docs. WebSocket API. https://developer.mozilla.org/en-US/docs/Web/API/WebSocket
14. W3C. The WebSocket API. https://websockets.spec.whatwg.org/
15. Lucide. Lucide documentation. https://lucide.dev/

Project-specific implementation claims are based on the supplied source files: pom.xml, package.json, application.properties, Java controllers/entities/configuration/services, React components, hooks and API modules.

Plagiarism Report

The supplied source report ends with a plagiarism report page, but no originality report or similarity result was provided with the project files. No score, certificate, approval or institutional finding has been fabricated.

Attach the institution-issued originality report on this page before submission. This placeholder is editable and preserves the final-page position of the supplied report.

Institution-issued report to be inserted here.